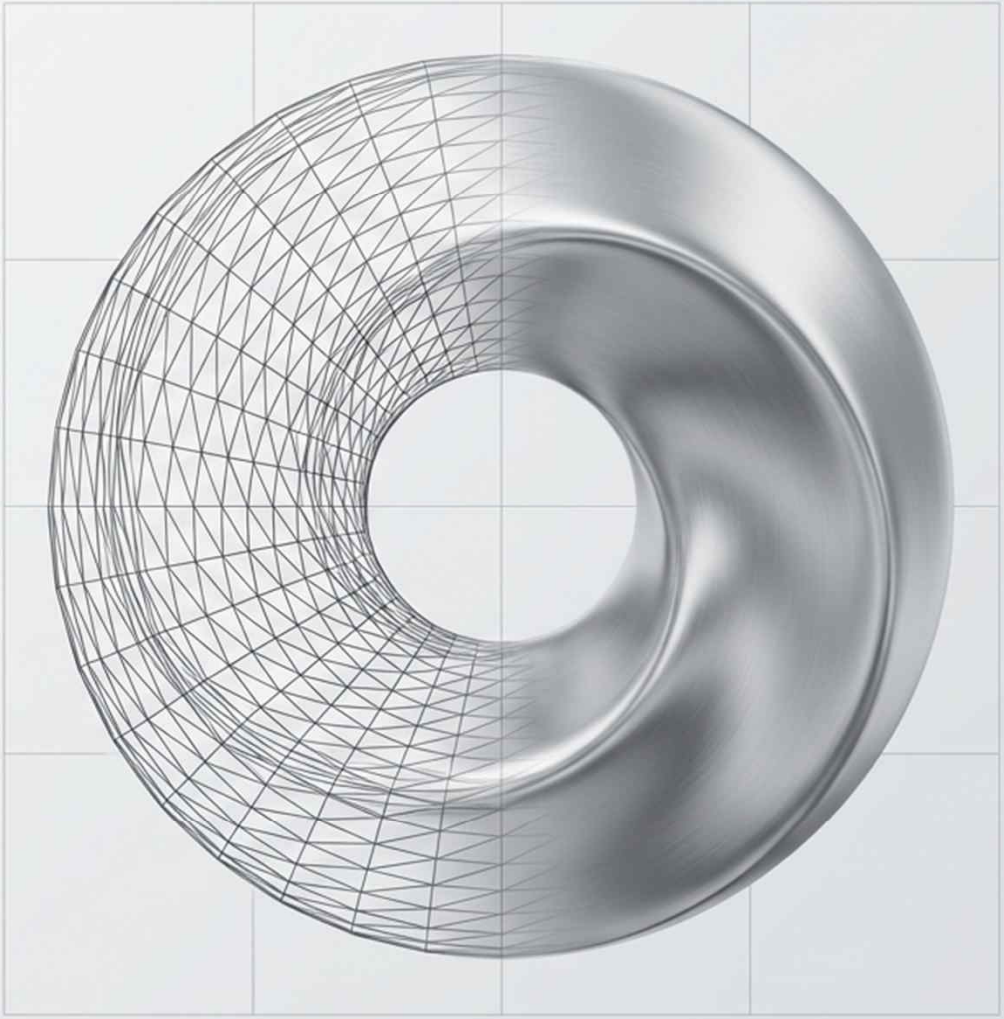
03/09

2026년 셰이더프로그래밍

한국공학대학교 게임공학과

이택희



한국공학대학교
TECH UNIVERSITY OF KOREA

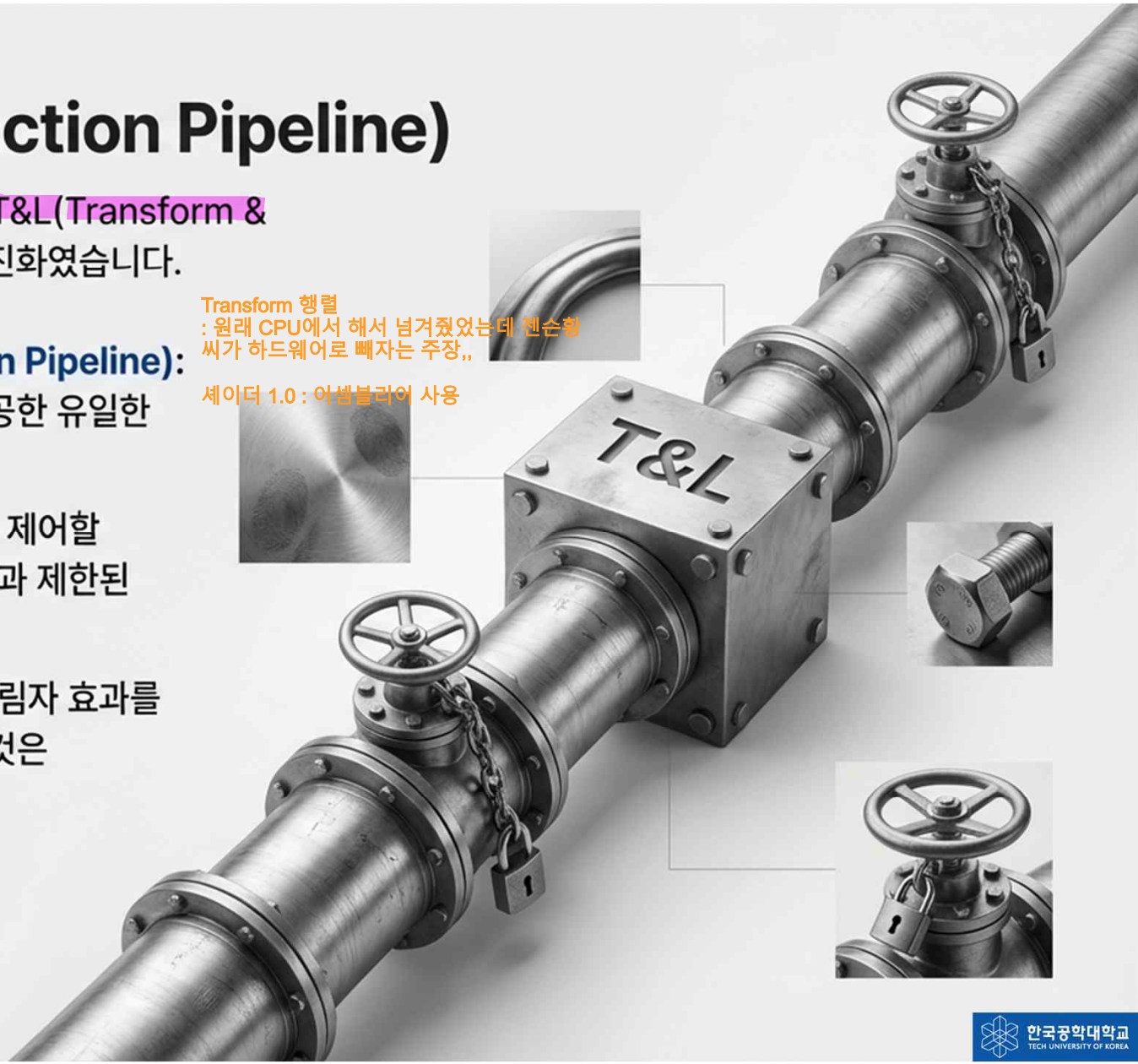
고정된 세계 (Fixed-Function Pipeline)

1999년 GeForce 256의 등장으로 하드웨어 T&L(Transform & Lighting)이 도입되었지만, 이는 한계를 가진 진화였습니다.

- **고정 그래픽스 파이프라인(Fixed-Function Pipeline):** 초기 그래픽스 API와 비디오 하드웨어가 제공한 유일한 렌더링 파이프라인.
- **통제력의 부재:** 개발자는 렌더링 과정을 직접 제어할 수 없었습니다. 하드웨어에 미리 정의된 공식과 제한된 옵션에 의존해야만 했습니다.
- **커스텀 알고리즘 불가:** 독창적인 조명이나 그림자 효과를 위한 자체 알고리즘을 하드웨어에 이식하는 것은 불가능했습니다.

Transform 행렬
: 원래 CPU에서 해서 넘겨줬었는데 젠슨황 씨가 하드웨어로 빼자는 주장,,

셰이더 1.0 : 어셈블러어 사용



프로그래밍의 시대가 열리다

2001년, NVIDIA GeForce 3(NV20)의 출시는 '설정(Configuration)'에서 '프로그래밍(Programming)'으로의 패러다임 전환을 의미했습니다.

- **최초의 프로그래머블 셰이더 도입:** 하드웨어에 고정되어 있던 파이프라인의 핵심 단계들이 개방되었습니다.
- **버텍스 프로그램 (Vertex Program):** 개발자가 직접 3D 정점의 위치, 색상, 방향을 수학적으로 조작할 수 있게 되었습니다. Skinning, Height Map
- **자유도의 획득:** 정해진 공식이 아닌, 개발자가 작성한 작은 프로그램(Shader)이 GPU 내에서 직접 실행되며 3D 그래픽스의 한계를 확장하기 시작했습니다.

병목의 극복: 통합 셰이더 아키텍처

점점 복잡해지는 셰이더 알고리즘은 기존의 분리된 파이프라인(버텍스, 픽셀 셰이더의 물리적 분리)에서 심각한 병목 현상을 유발했습니다.

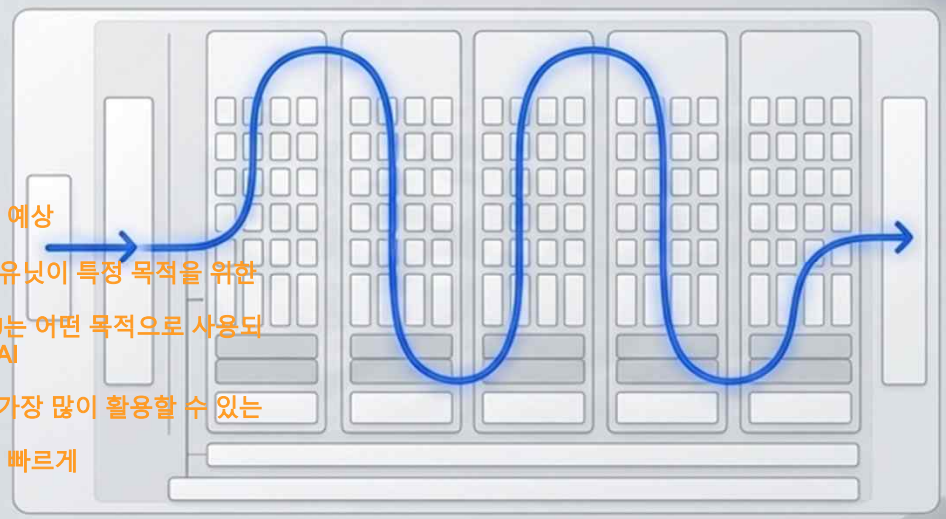
연산량

- **GeForce 8800 (2006년)**: DirectX 10 세대를 선도한 혁명적인 구조.
- **Unified Shader Architecture:** 버텍스와 프래그먼트 셰이더의 명령어를 단일 명령어 세트로 병합했습니다.
- **Streaming Multiprocessors (SM)**: 128개의 유연한 처리 장치가 작업의 종류에 상관없이 동적으로 할당되어 프로세서의 효율을 극대화 했습니다. 이것이 현대 GPGPU(범용 GPU) 연산의 시발점이 되었습니다.

GPU가 그래픽 렌더링용으로만 쓰이지는 않을 것을 예상

GPU의 연산 유닛이 특정 목적을 위한 것이 아님
–> 지금 GPU는 어떤 목적으로 사용되고 있는가? : AI

AI가 GPU를 가장 많이 활용할 수 있는 이유는?
: 단순 연산을 빠르게



파이프라인을 흐르는 것은 무엇인가?

이 거대한 그래픽스 파이프라인과 수천 개의 프로세서를 거쳐
화면의 픽셀로 변환되는 '원시 데이터(Raw Data)'의 정체.

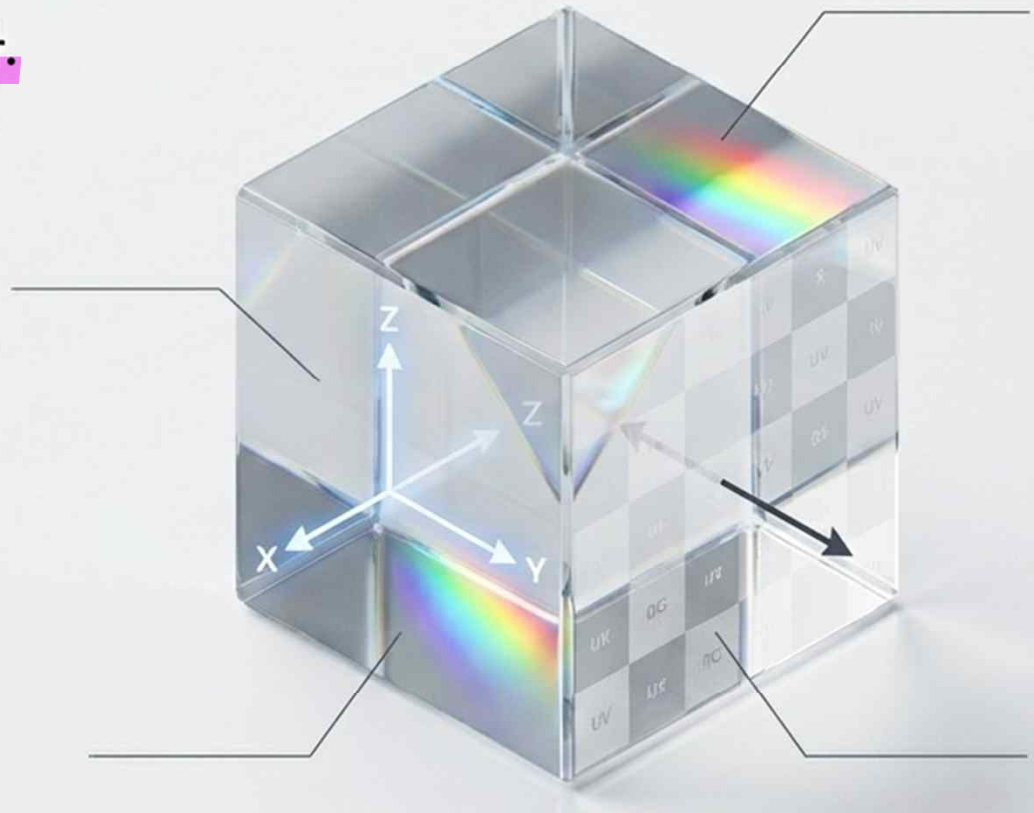
병렬처리를 위한 정보의 조각들

그래픽스 파이프라인의 가장 근본적인 주인공, 정점(Vertex)에 대해
우리가 가진 고정관념을 깰 차례입니다.

정점(Vertex): 의미를 가진 정보의 집합

정점은 단순히 3차원 공간의 점 (x, y, z)가 아닙니다.
렌더링을 위한 모든 속성을 캡슐화한 풍부한 데이터 패킷입니다.

- **Position (위치):** 3D 공간 상의 위치 좌표 (x, y, z, w)
- **Color (색상):** 정점이 지닌 고유의 색상 값 (R, G, B, A)
- **Normal (노멀):** 빛의 반사를 계산하기 위한 표면의 수직 벡터 (nx, ny, nz)
- **Texture Coordinates (텍스처 좌표):** 2D 이미지를 입히기 위한 좌표 (U, V)



이 데이터 패킷이 파이프라인을 통과하며
모니터 화면의 최종 픽셀로 진화합니다.

3D 그래픽스 렌더링 파이프라인 개요

원시 데이터가 모니터의 **픽셀**이 되기까지, 각 단계의 출력은
다음 단계의 **입력**이 되는 순차적 여정을 거칩니다.

* Pipeline이란

앞선 단계의 출력을 입력으로 받음
넘겼으면 서로 종속되지 않고 데이터에
대해 완전히 독립

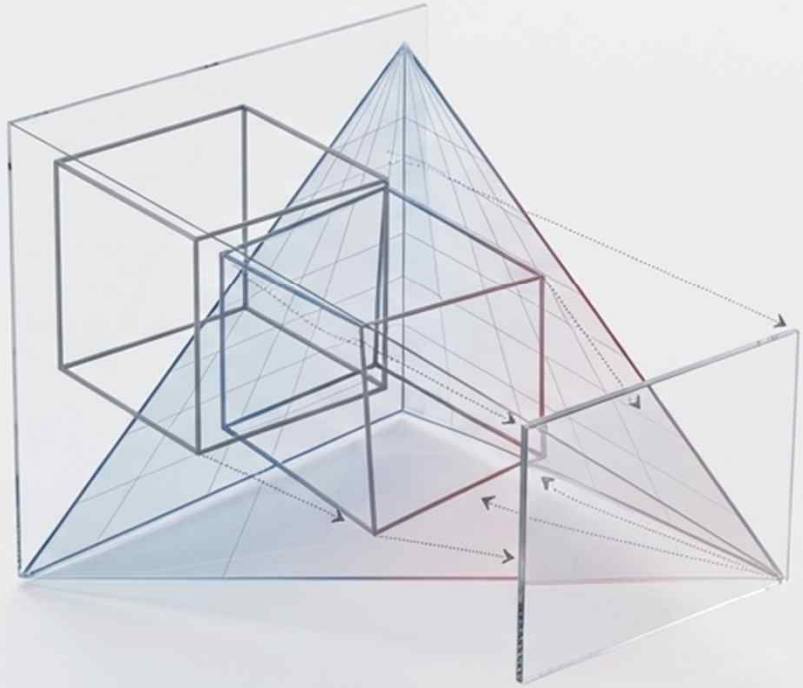
두 번째 단계로 넘겼으면 첫 번째 애는
새로운 데이터를 받을 수 있음.

Stage 1. 버텍스 셰이더 (Vertex Shader)

외부에 있는 데이터를 전달받는 첫 번째 입구
버텍스 셰이더의 의미: 입구의 역할

가장 먼저 **데이터 패킷**(Vertex)을 맞이하는 프로그래머블 단계입니다.

- **공간의 변환**: 오브젝트 공간(Object Space)에 있던 3D 정점들을 카메라의 시점과 화면 비율에 맞춘 **화면 공간**(Screen Space / Clip Space)으로 변환합니다.
- **속성 조작**: 이 과정에서 필요에 따라 정점의 위치, 색상, 노멀 방향을 수학적으로 조작합니다.
- **독립적 연산**: 각 정점 데이터는 완전히 독립적이므로, GPU의 수많은 코어에서 완벽하게 병렬로 처리됩니다.



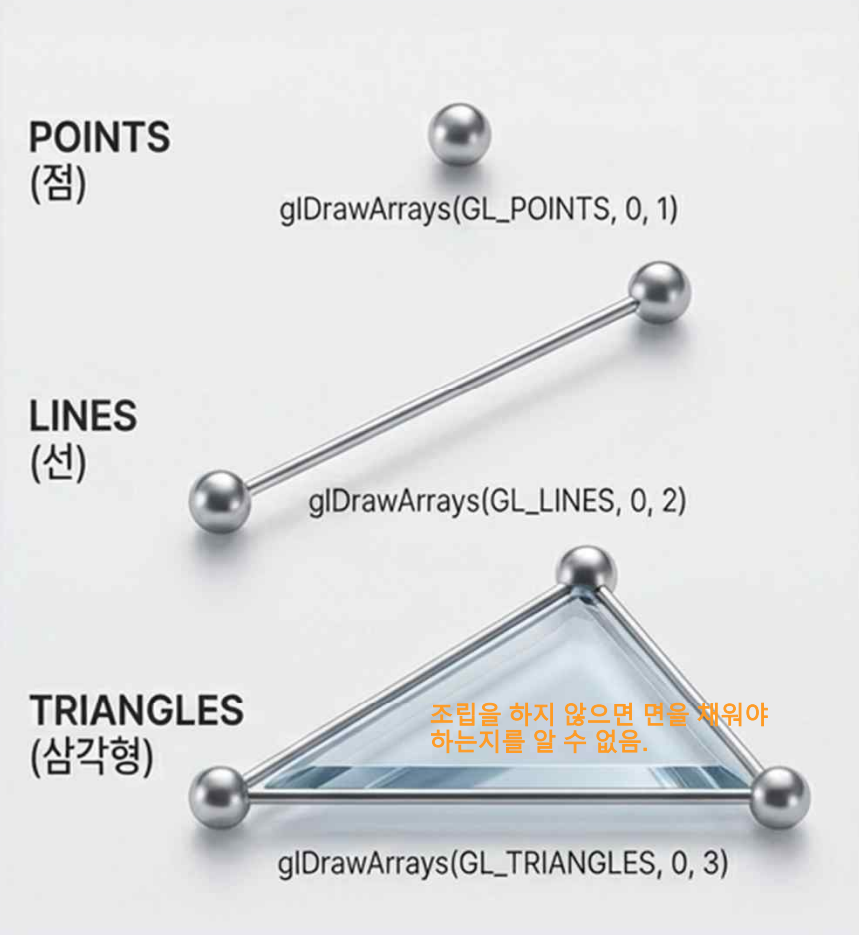
Space Transformation

Stage 2. 원시 도형 조립 (Primitive Assembly)

파이프라인을 통과한 개별 정점들은 서로 연결되어 의미 있는 3D 기하학적 기본 도형(Primitive)으로 형태를 갖춥니다.

- **Points (점)**: 단일 정점 glDrawArrays(GL_POINTS, 0, 1)
- **Lines (선)**: 두 개의 정점 연결 glDrawArrays(GL_LINES, 0, 2)
- **Triangles (삼각형)**: 세 개의 정점이 연결되어 형성된 3차원 공간 상의 '면' glDrawArrays(GL_TRIANGLES, 0, 3)

이 단계에서 비로소 3D 오브젝트의 뼈대가 완성됩니다.



Stage 3. 래스터화 (Rasterization)

벡터 기반의 3D 삼각형이 픽셀 기반의 2D 디스플레이 세계로 진입하는 결정적 순간입니다.

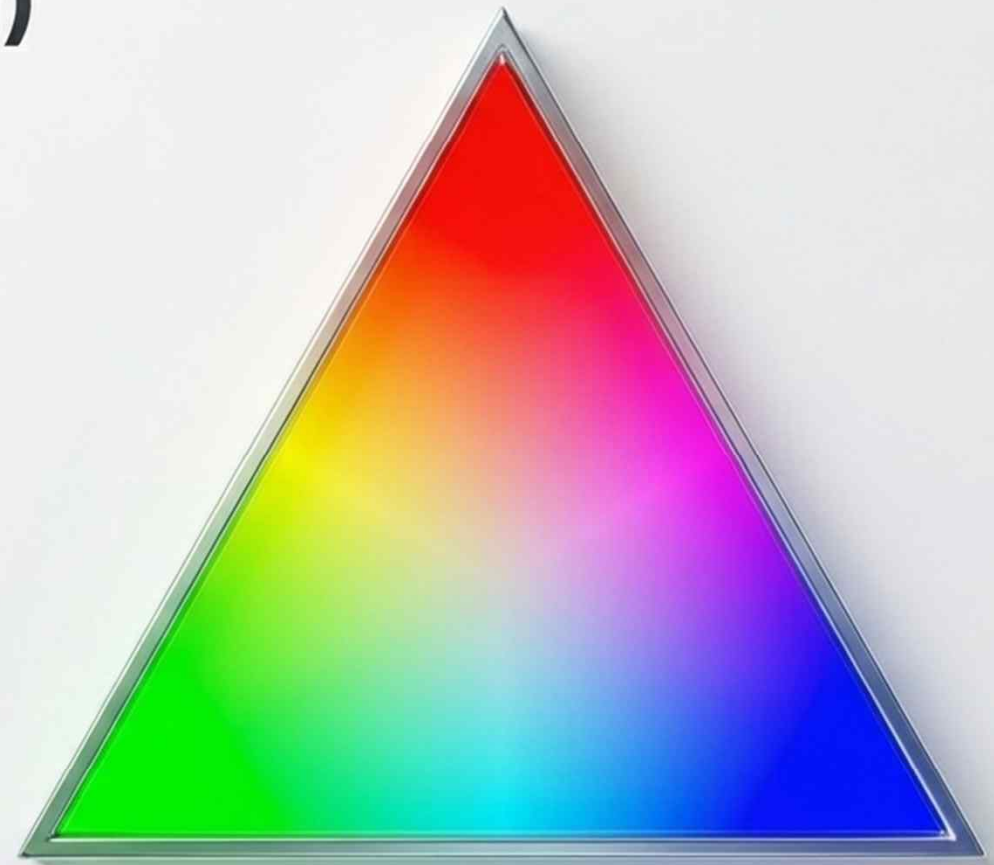
- **조각내기**: 조립된 삼각형(Primitive)을 래스터 스캔하여 모니터의 **픽셀 격자(Grid)**에 정확히 들어맞는 2차원 조각들로 분할합니다.
- **프래그먼트(Fragment)의 탄생**: 분할된 이 작은 조각들을 프래그먼트라고 부릅니다. 이들은 예비 픽셀로서 화면에 그려질 자격을 얻게 됩니다.

보간의 마법 (Interpolation)

버텍스라는 개념은 이 단계부터 없어지고, 픽셀로 소통

세 개의 정점으로 만들어진 삼각형 내부의 빈 공간은 어떻게 채워질까요?

- 정점이 가진 **'의미 있는 정보'**(위치, 색상, 노멀, 텍스처 좌표)는 래스터화 과정에서 파괴되지 않습니다.
- **부드러운 혼합:** 삼각형 내부의 수많은 프래그먼트들을 생성할 때, 3개 정점의 데이터 값이 거리에 비례하여 수학적으로 부드럽게 섞여(Bilinear interpolation 등) 각 조각에 할당됩니다.
- 단 3개의 색상 데이터만으로도 완벽하고 자연스러운 그라데이션 면이 형성되는 원리입니다.



Stage 4. 프래그먼트 셰이더 (Fragment Shader)

화면에 그려질 위치는 버텍스 셰이더가 결정, 컬러는 프래그먼트 셰이더가 결정한다.

래스터화를 통해 생성된 수많은 프래그먼트 조각들에
최종적인 생명(색상)을 불어넣는 프로그래머블 단계입니다.

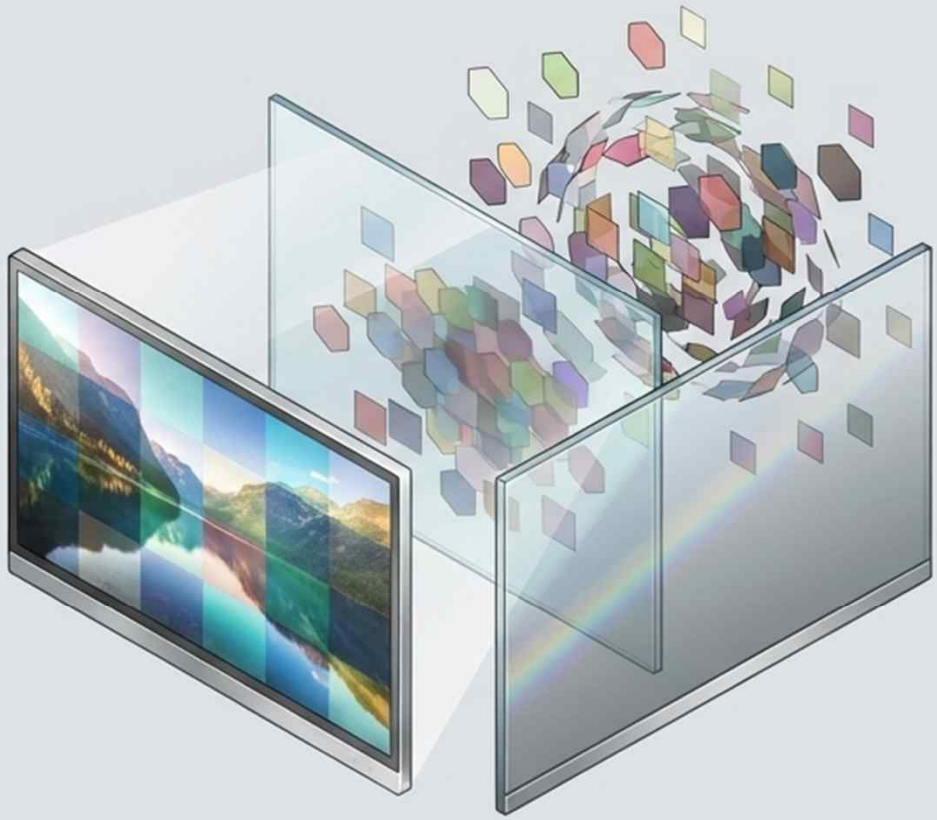
- **픽셀 단위 셰이딩:** 이전의 고정 파이프라인에서는 불가능했던 정교한 **픽셀 단위**(Per-fragment)의 조명 계산을 수행합니다.
- **질감(Texture) 맵핑:** 보간된 텍스처 좌표(UV)를 바탕으로 2D 이미지 데이터를 읽어와 질감을 입힙니다.
- **RG/BA 도출:** 연산의 최종 결과물로 각 픽셀 샘플 위치에 반영될 최종적인 부동소수점 **RGBA 색상 값**을 산출합니다.

Stage 5. 프레임 버퍼 (Frame Buffer & Output Merging)

GPU메모리에 할당

길었던 데이터의 여정이 끝나고, 우리가 실제로 보는 이미지가 완성되는 최종 단계입니다.

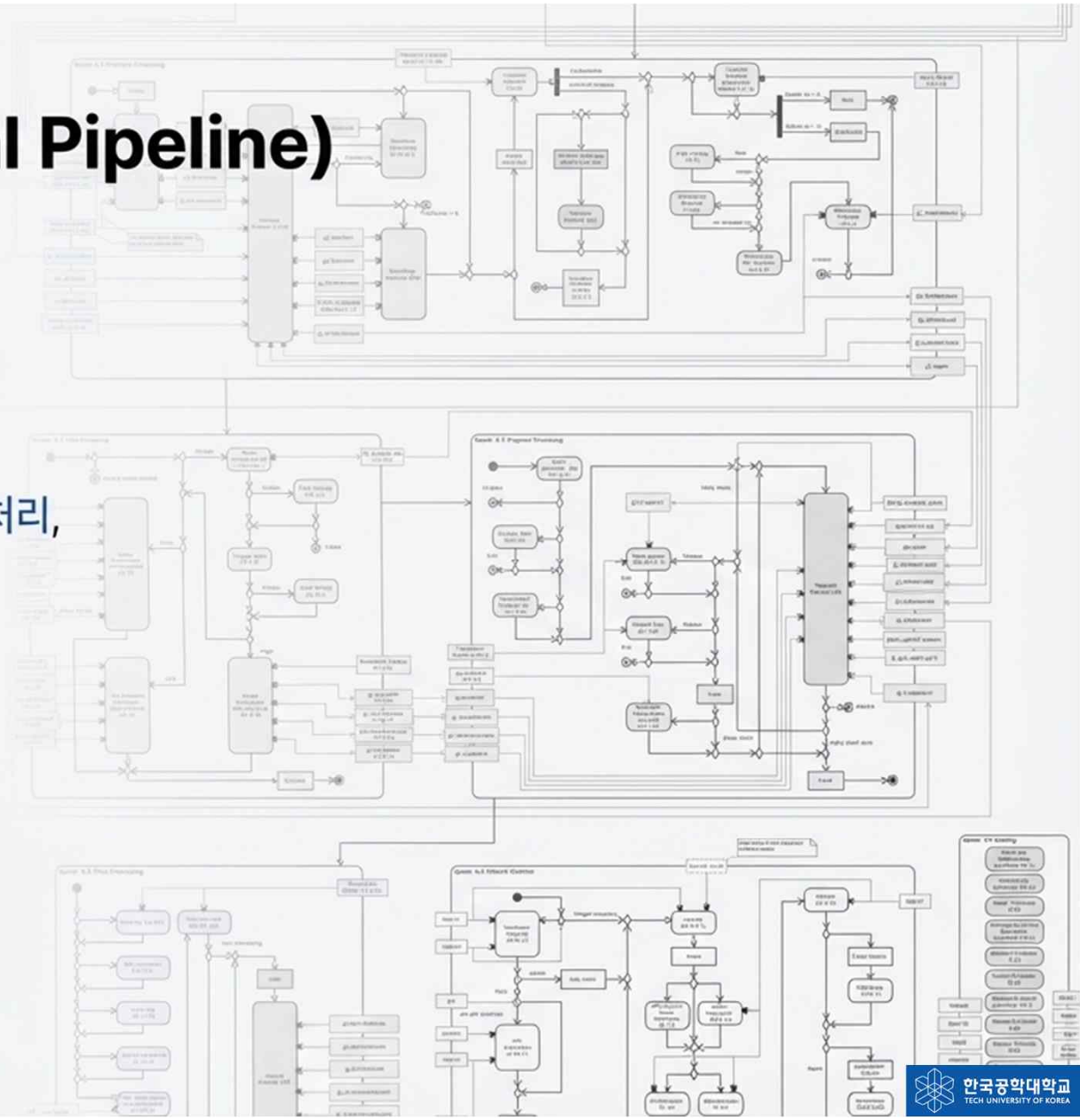
- **출력 병합 (Output Merging):** 3D 공간에서 렌더링된 수많은 프래그먼트들의 색상과 깊이(Depth) 값 등을 비교 및 병합합니다.
- **2D 픽셀 배열:** 살아남은 데이터들은 2D 형태의 색상 픽셀 배열로 최종 변환됩니다.
- **Monitor 출력:** 이 데이터가 프레임 버퍼(Frame Buffer)에 저장된 후, 디스플레이 장치로 전송되어 비로소 우리의 눈앞에 놀라운 3D 렌더링 화면으로 나타납니다.



현실의 파이프라인 (The Real Pipeline)

우리가 살펴본 논리적 단계들은 방대하고 복잡한
실제 시스템의 핵심 뼈대입니다.

현재의 OpenGL 4.3 기반 그래픽스 파이프라인은
정점 제어, 테셀레이션(Tessellation), 지오메트리 처리,
트랜스폼 피드백 등 수십 개의 세부 단계가 정교하게
맞물려 돌아가는 거대한 엔지니어링의
결정체입니다.



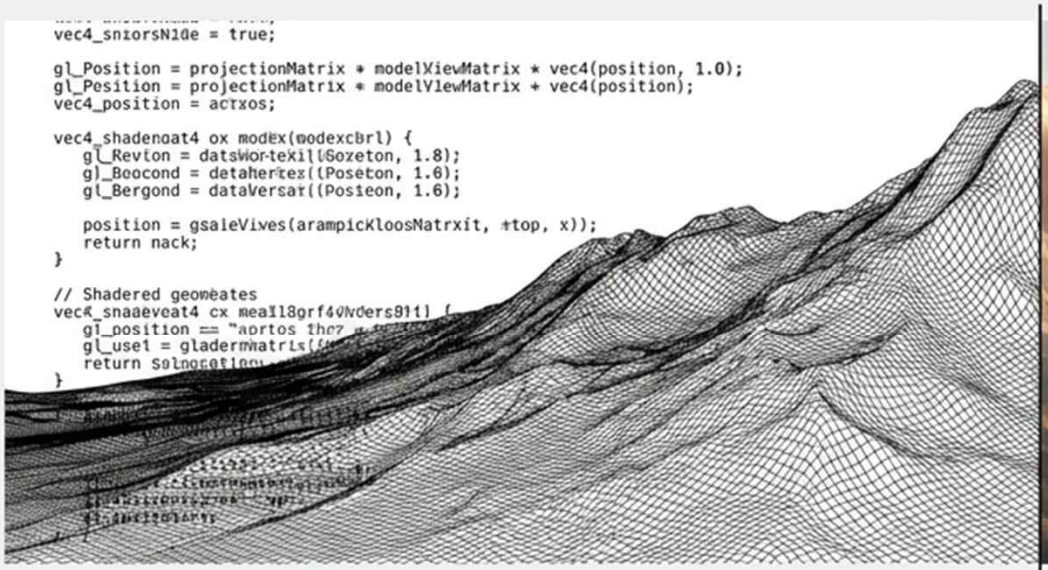
놀라운 아름다움 뒤에 숨겨진 복잡성

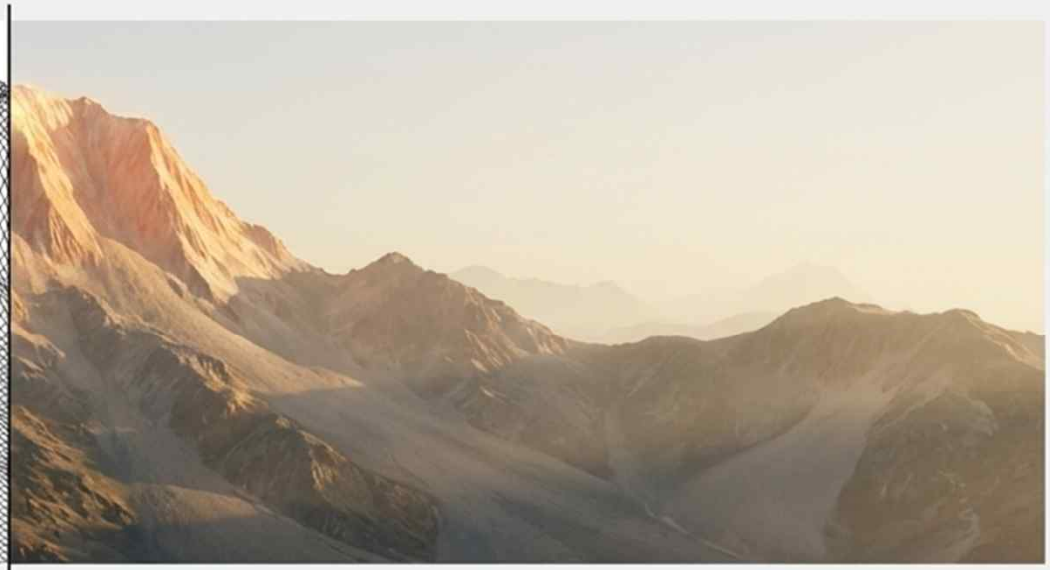
데이터의 여정 요약:

고정된 파이프라인의 제약을 벗어나기 위한 갈망은 프로그래머블 셰이더(GeForce 3)를 탄생시켰고,
더 높은 효율을 향한 집념은 통합 셰이더 아키텍처(GeForce 8800)를 완성했습니다.

단순한 좌표가 아닌 '풍부한 정보의 집합(Vertex)'이 이 유연하고 강력한 하드웨어를 통과하며,
초당 수십억 번의 변환과 보간을 거쳐 픽셀로 재탄생합니다.

우리가 화면에서 마주하는 경이로운 3D 그래픽은 바로 이 데이터와 하드웨어의 진화가 만들어낸 완벽한 조화의 결과입니다.





"이 수업은 기본으로 부터 시작하여
복잡한 렌더링 결과를 만들어내는 과정을 경험합니다.
기본은 여러분들의 머리속에 정착되어야 하고,
그 정착된 지식을 기반으로 AI와 소통하며
더 나은 결과를 만들어 낼 것입니다. 잘 진행해 봅시다."

- 이택희, 2026년 세이더프로그래밍 시작하며 -

이틀과 6개월의 차이는? 기본.
공부를 열심히 해서 시험을 잘 보자.